Software Development Report

An investigation into Lightweight Hand Pose estimation for Edge HCI

Brandon MacGillivray

40321491

Acknowledgements

Contents

1 Purpose, Operation and Validation **2**

2 Design and Implementation **2**

2.1 Significant Assumptions Made for the Design and Implementation 3
2.2 Languages, Packages, Frameworks, and Libraries Used 4
2.3 Overall Architecture / System Design and Interfaces Between Components . . . 4
2.4 Data Model(s), Including In-Memory Structures and Persistent Storage 8
2.5 Significant Methods, Functions, or Other Lower-Level Elements 8
2.6 Important Errors and Their Conditions 10
2.7 User Interface Design 10
2.8 Quality Assurance Steps 11
2.8.1 Version Control and Versioning 11
2.8.2 Code Style and Convention 11
2.8.3 Automated Testing 12

1 Purpose, Operation and Validation

The software is intended to implement and evaluate a lightweight 2D hand pose estimation pipeline for edge HCI, with a DRHand-inspired dual-branch model that predicts hand keypoints from RGB images and supports training, inference, benchmarking, and reproducible experiment reruns.

The source repository for this software is available at `https://github.com/Brandon-MacGillivray/CSC4006_research_software.git`.

The primary persona for this software is a researcher working on lightweight hand pose estimation who needs to train, evaluate, and reproduce experiments from the repository.

The main user stories for this software are:

- "As a researcher, I want to train the DRHand-inspired model on the supported datasets so that I can study its behaviour under different configurations."
- "As a researcher, I want to evaluate saved checkpoints and compare metrics so that I can judge model accuracy and robustness."
- "As a researcher, I want to rerun experiments from documented commands and preserved artefacts so that I can verify that the reported results are reproducible."
- "As a researcher, I want to run inference and generate qualitative outputs so that I can inspect the predicted hand keypoints visually."

A user operates the software from the command line by setting up the documented Python environment, then running the repository scripts for the required workflow, such as `scripts/train.py` for training, `scripts/eval_metrics.py` for checkpoint evaluation, and `scripts/predict_image.py` for single-image inference, with inputs and options supplied as command-line arguments.

The software takes inputs such as dataset roots, checkpoint files, single RGB images, model and evaluation settings passed through command-line arguments, and it produces outputs such as trained checkpoint files, evaluation JSON metrics, aggregated CSV result tables, benchmark summaries, and qualitative prediction overlays or rendered figures.

A user can tell the software is working correctly when the lightweight test suite passes, the main command-line scripts load and print valid help output, and training, evaluation, inference, or benchmarking workflows execute successfully and produce outputs in the expected format that can be compared against the preserved reference JSON, CSV, checkpoint, and qualitative artefacts bundled in the repository.

2 Design and Implementation

The software is implemented as a modular research artefact whose main design goal is to separate reusable hand-pose functionality from workflow-specific execution logic. Core behaviour is kept in the Python package under `src/handpose`, while command-line scripts under `scripts/` provide thin entry points for training, evaluation, inference, benchmarking, aggregation, and qualitative rendering. This allows the same implementation to support day-to-day experimentation, controlled reruns, and assessment-oriented replication without duplicating logic across multiple workflows.

Within the package, responsibilities are separated by concern rather than by script. The `handpose.data` modules provide a common interface over the supported datasets and keypoint layouts, `handpose.models` defines the DRHand-inspired dual-branch network

and its losses, `handpose.training` contains the stage-specific optimisation and artefact logging code, `handpose.inference` implements prediction and branch-fusion behaviour, `handpose.evaluation` computes metrics and result payloads, and `handpose.checkpoints` standardises checkpoint persistence and metadata validation. This structure was intended to keep the codebase easier to test, extend, and reuse across both the full 21-keypoint configuration and the reduced 10-keypoint experiments.

At implementation level, the system is built around a small set of consistent internal representations: resized RGB inputs, normalized keypoint coordinates, metadata-rich checkpoints, and structured JSON or CSV outputs. Dataset loaders convert dataset-specific annotations into a shared format, the model produces heatmap and coordinate predictions, fused inference combines these branches when required, and the surrounding scripts orchestrate complete workflows such as training, evaluation, benchmarking, and result aggregation. As a result, reproducibility is treated as part of the software design itself rather than as an external add-on, with experiment definitions, preserved checkpoints, and reference artefacts integrated directly into the repository.

2.1 Significant Assumptions Made for the Design and Implementation

The software assumes that the user has installed the supporting software required for the intended workflow, including core dependencies and any optional packages as defined in the documented installation and replication instructions.

The software assumes that the user has access to the required datasets or preserved artefacts in the directory structure expected by the repository scripts.

The software assumes that the user has sufficient hardware resources for the intended task. Smaller inference checks may be possible on CPU, but CUDA GPU acceleration is strongly recommended for training and larger evaluation runs.

Framework equivalence. The software assumes that the DRHand method can be reimplemented in PyTorch even though the original paper describes a TensorFlow implementation. As a result, this repository should be understood as a DRHand-inspired reconstruction rather than a strict reproduction, so exact numerical agreement with the original paper should not be expected.

Heatmap supervision reconstruction. The software assumes Gaussian target heatmaps for heatmap regression because the paper defines an L2 heatmap loss but does not specify exactly how target heatmaps are constructed. This affects optimisation because the heatmap branch is trained to match smooth Gaussian peaks rather than some other target representation, which changes convergence behaviour.

Effective batch-size reconstruction. The software assumes that the paper's stage-2 batch size of 256 can be approximated using batch size 64 with gradient accumulation on different hardware. This makes the training pipeline usable on more limited systems, but it also means the repository reproduces the paper's intended effective batch size only approximately rather than literally.

No explicit augmentation policy. The software assumes a simple deterministic preprocessing pipeline because the paper does not specify a detailed augmentation strategy.

Visibility-aware evaluation assumption. The software assumes that evaluation should mask invisible joints when visibility information is available. This affects the reported metrics because performance is measured primarily on visible landmarks, which may differ from alternative evaluation protocols. This is a consequence of the annotation specifics of the RHD dataset. Where often the coordinates of hands outside the image frame are still recorded.

2.2 Languages, Packages, Frameworks, and Libraries Used

Python. The software was built primarily in Python because it provides a practical and widely used language for machine-learning research, rapid experimentation, data processing, and command-line workflow development. This made it suitable for implementing a research artefact that needed to support training, evaluation, inference, benchmarking, and reproducibility.

MediaPipe. MediaPipe was used because the project included an explicit baseline comparison against a strong lightweight hand-tracking system and also relied on MediaPipe-related assets for pseudo-labelled or baseline-oriented workflows. This choice was important because it gave the project a practical reference point against a widely used real-time hand-pose system.

PyTorch. The main hand-pose pipeline was implemented in PyTorch because the project required a flexible deep-learning framework for defining custom model architectures, losses, and training procedures. PyTorch was particularly suitable for this DRHand-inspired implementation because it allowed the original dual-branch idea to be reconstructed clearly while remaining easy to adapt for the reduced 10-keypoint and transfer-learning experiments.

PyTest. PyTest was used for automated testing because the repository needed a lightweight way to verify important behaviour such as model output shapes, fusion logic, and checkpoint validation. It was chosen to support software quality assurance and to provide a quick validation route for assessors and future users of the artefact.

2.3 Overall Architecture / System Design and Interfaces Between Components

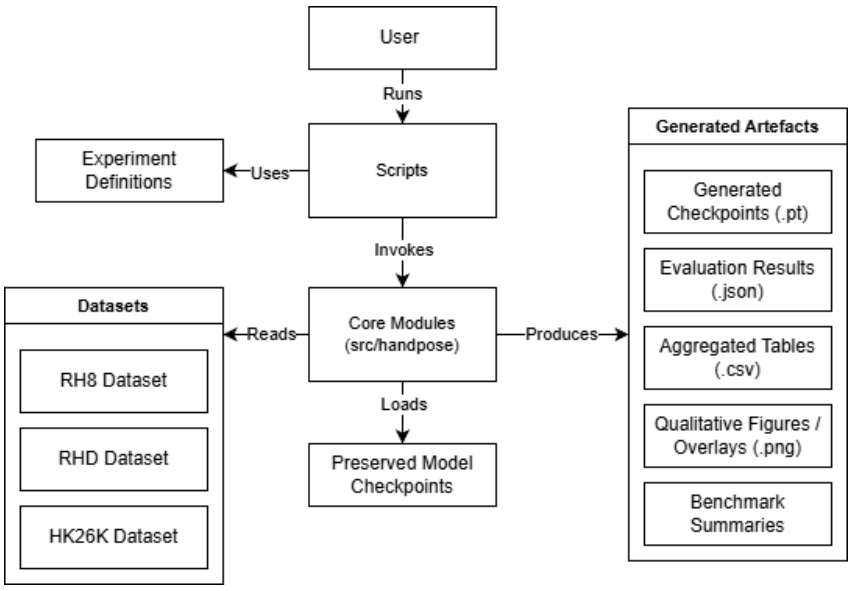

Figure 1: High-level architecture.

Overall Structure

Figure 1 shows the overall system structure. The repository is designed as a modular research artefact in which thin command-line scripts provide the user-facing workflows, while the reusable implementation is contained in the `src/handpose/` package. This separates workflow orchestration from the core hand-pose logic and allows the same internal code to support training, evaluation, benchmarking, and single-image inference.

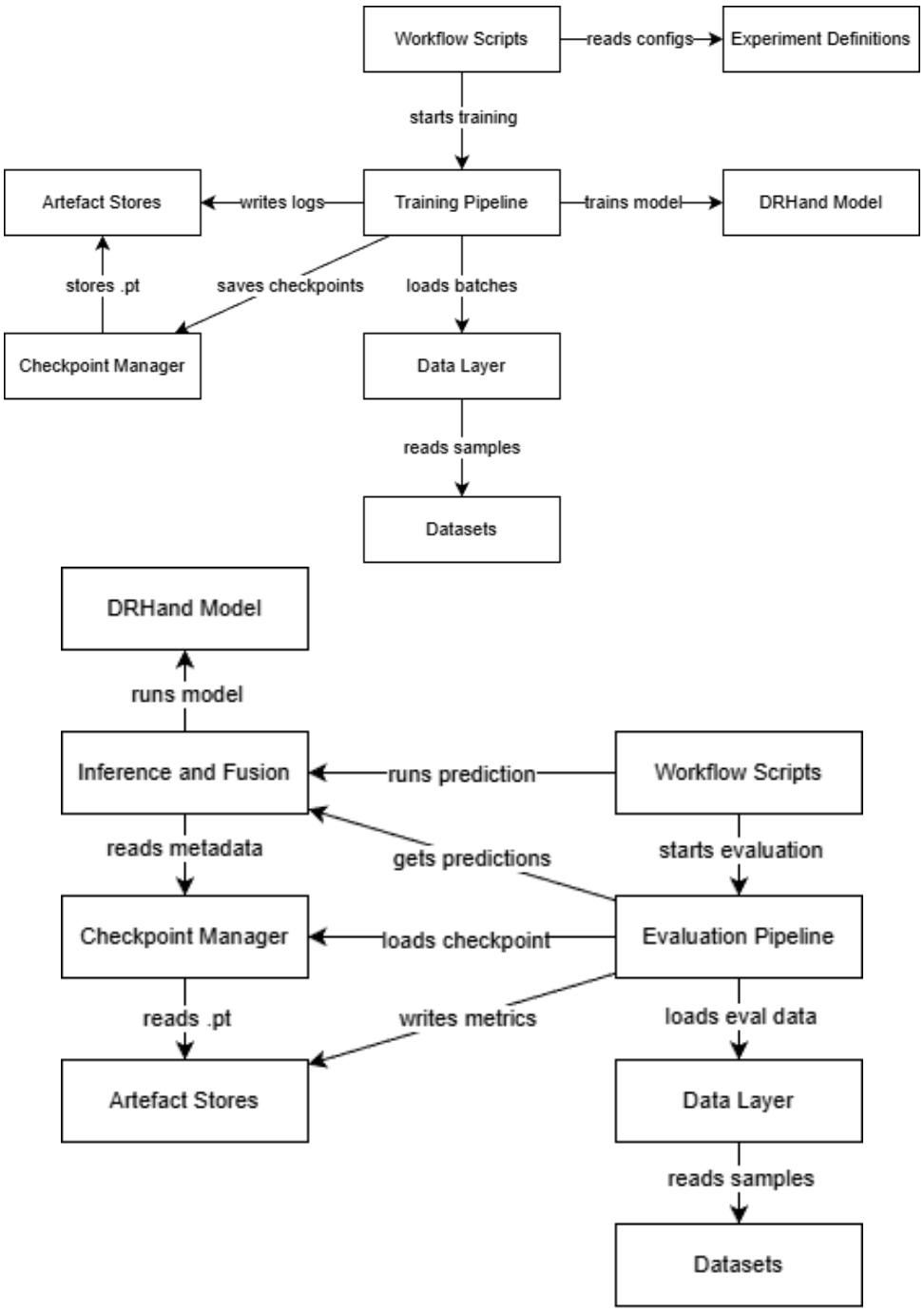

Figure 2: Training and evaluation component views.

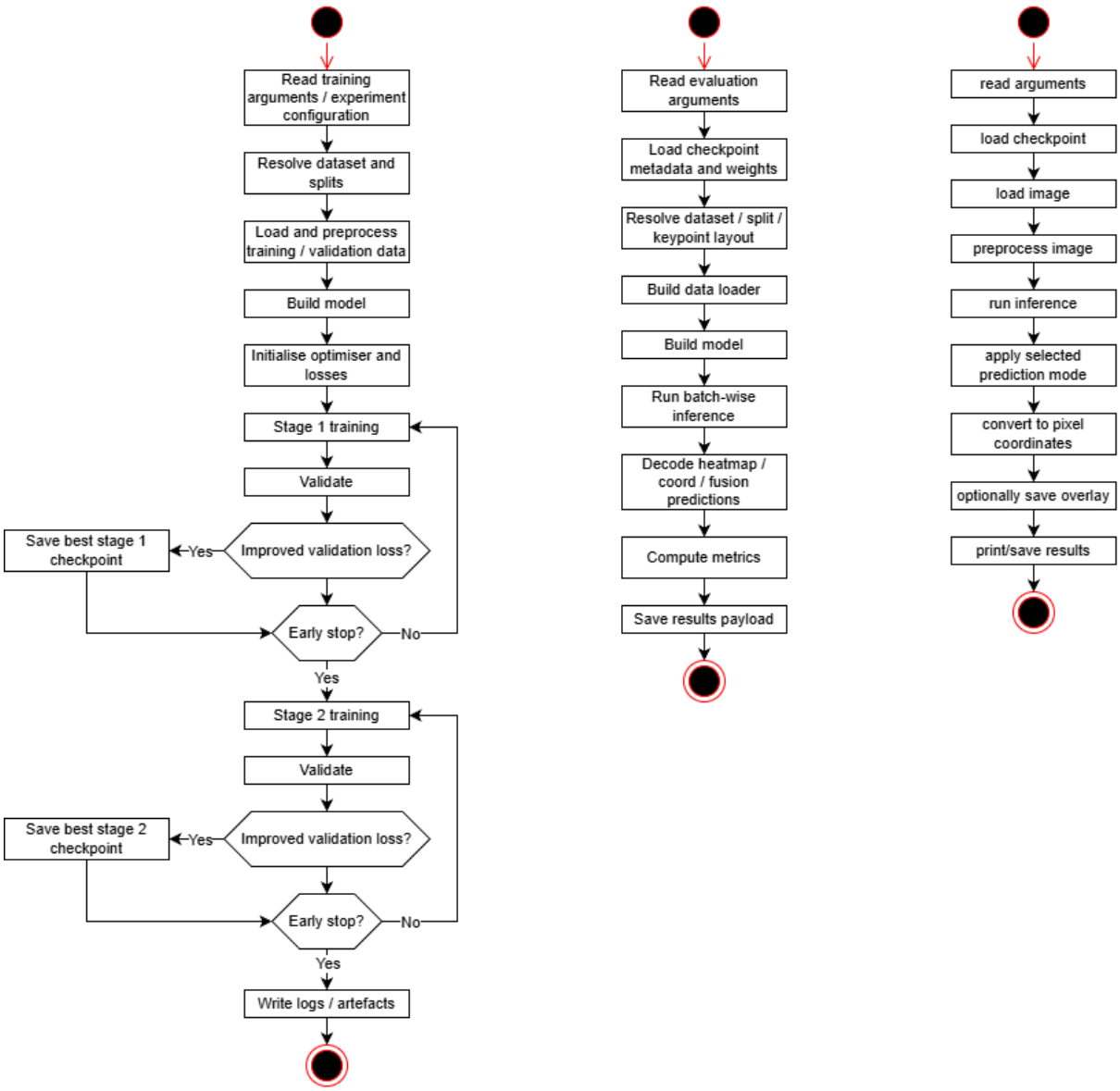

Figure 3: Training, evaluation and inference activity diagrams.

At the highest level, the user interacts with the software by running scripts such as `scripts/train.py`, `scripts/eval_metrics.py`, and `scripts/predict_image.py`. These scripts optionally use experiment definitions and generated command files, then invoke the core modules responsible for data loading, model execution, training, inference, evaluation, and checkpoint handling.

The core modules read dataset files from disk and produce new artefacts such as trained checkpoints, evaluation JSON files, aggregated CSV tables, benchmark summaries, and qualitative figures. This structure keeps the execution interface simple while ensuring that the main implementation remains reusable and consistent across workflows.

Training View

Figure 2 expands this design into separate training and evaluation component views. In the training view, the workflow scripts start the training pipeline, which requests batches from the data layer, passes them through the DRHand model, computes the required losses, and updates the model parameters.

The data layer provides a shared interface for loading and preprocessing supported datasets into normalized tensor representations. During training, the checkpoint manager validates and saves model state together with metadata, while the training pipeline also writes logs and loss histories to the artefact store.

Evaluation and Inference View

In the evaluation and inference view, the workflow scripts start either checkpoint evaluation or direct prediction. The evaluation pipeline loads a saved checkpoint through the checkpoint manager, rebuilds the model configuration, obtains evaluation samples from the data layer, and requests predictions from the inference and fusion module.

That module runs the DRHand model, decodes heatmap outputs, uses the coordinate branch predictions, and applies the fusion rule when fused inference is requested.

The resulting predictions are then used by the evaluation pipeline to compute metrics such as SSE, EPE, PCK, timing information, and optional fusion diagnostics, which are written back to the artefact store as structured outputs.

Workflow Activities

Figure 3 complements the component views by showing the workflows as activities. It emphasizes the execution order of the main repository tasks: training moves from dataset loading and preprocessing to staged optimisation, validation, and checkpoint export, while evaluation and inference move from checkpoint loading and sample preparation to prediction, fusion, metric computation, and output generation.

Interfaces Between Components

The interfaces between these components are mainly file-based and tensor-based. File-based interfaces include dataset roots, annotation files, checkpoint `.pt` files, evaluation `.json` outputs, summary `.csv` files, and rendered `.png` figures.

In-memory interfaces are primarily PyTorch tensors representing images, keypoint coordinates, visibility values, heatmaps, and predicted coordinates. Checkpoint metadata provides an additional interface between workflows by preserving information such as training stage, epoch, keypoint layout, and other configuration details needed to reload and evaluate a model correctly.

2.4 Data Model(s), Including In-Memory Structures and Persistent Storage

Processed Data

The main inputs are RGB hand images and per-image annotations from the supported datasets, including 2D keypoint coordinates and visibility values. During preprocessing, these inputs are converted into a shared tensor representation that is reused across training, evaluation, and inference.

In-Memory Representation

The main in-memory structures are image tensors of shape `(C,H,W)`, keypoint-coordinate tensors of shape `(K,2)`, visibility tensors of shape `(K)`, predicted heatmaps of shape `(K,64,64)`, and predicted coordinate tensors of shape `(K,2)`.

Where C denotes the number of image channels, H and W denote image height and width, K denotes the number of keypoints, and N denotes batch size. The model uses RGB inputs, so typically C = 3, and the predicted heatmaps have spatial size 64 x 64.

Persistent Storage

Persistent storage in the repository is file-based. Source datasets remain on disk as image and annotation files, while intermediate and final outputs are saved as experiment artefacts. Training checkpoints are written as PyTorch `.pt` files containing model state and metadata, loss histories are stored as CSV, evaluation outputs are written as JSON, aggregated summaries are stored as CSV, and qualitative overlays or rendered figures are exported as `.png`. This design supports reproducibility because trained states, metrics, and generated figures can all be reloaded or compared directly from the filesystem.

Data item	**In-memory form**	**Persistent form**
RGB input image	Preprocessed tensor `(3,H,W)`	`.png` or `.jpg` image files
Ground-truth hand pose	Keypoint tensor `(K,2)` plus visibility tensor `(K)`	RHD annotations or COCO-style `.json` annotations
Training / evaluation batch	Batched tensors such as `(N,3,H,W)` and `(N,K,2)`	Runtime only; constructed by data loaders
Model predictions	Heatmaps `(N,K,64,64)` and coordinates `(N,K,2)`	Exported prediction or metric payloads in `.json`; optional qualitative `.png` outputs
Checkpoint state	Python dict containing `model_state`, keypoint layout, stage, epoch, and training metadata	PyTorch checkpoint `.pt` files
Result summaries	Scalar metrics and aggregated experiment statistics	CSV tables and JSON result files

Table 1: Main data representations used by the repository.

2.5 Significant Methods, Functions, or Other Lower-Level Elements

Core Model

The most important lower-level implementation element is `HandPoseNet` in `src/handpose/models/hand_pose_model.py`, which acts as the main wrapper around the learned model.

It combines the shared backbone, heatmap regression head, and coordinate regression head into one interface and provides the forward paths used during both training and inference. This makes it the central model component on which the rest of the repository depends.

Architecture and Optimisation

At the architectural level, the most significant lower-level classes are `Backbone`, `Heatmap_reg`, and `coord_reg` in `src/handpose/models/architecture.py`, together with the reusable block definitions in `src/handpose/models/block.py`.

These classes implement the shared feature extractor, the heatmap branch, the coordinate branch, and the depthwise-separable and upsampling building blocks from which the DRHand-inspired model is assembled.

The most important optimisation-related functions are `coords_to_heatmaps`, `HeatmapMSELoss`, and `WingLoss` in `src/handpose/models/losses.py`, along with `build_optimizer_stage1`, `build_optimizer_stage2`, `train_one_epoch`, and `validate` in the training modules.

These functions define how supervision targets are constructed, how the two training losses are computed, how different stages of the model are optimised, and how batch-wise training and validation are carried out.

Data Processing and Fusion

Another important group of lower-level functions appears in the preprocessing and dataset-loading code. The dataset factory and dataset classes in `src/handpose/data/` provide a unified interface for loading RHD and COCO-style hand datasets, while preprocessing helpers in `src/handpose/data/transforms.py` resize images, convert them to tensors, and rescale keypoint coordinates. These functions are significant because all training, evaluation, and inference workflows depend on the same shared input format.

The lower-level functions implementing branch fusion. In `src/handpose/inference/fusion.py`, functions such as `heatmaps_to_coords_argmax`, `resolve_fusion_bone_edges`, and `fuse_coords` define how heatmap predictions are decoded and how the final fused prediction is selected from the heatmap and coordinate branches. These functions are central because the fusion rule is one of the defining design features of the implemented pipeline.

Checkpointing and Workflow Entry Points

Checkpoint and evaluation handling. Functions in `src/handpose/checkpoints.py` define how checkpoints are validated, saved, and reloaded, while `evaluate_checkpoint` in `src/handpose/evaluation/eval_metrics_core.py` performs the main metric computation for SSE, EPE, PCK, timing, and optional fusion diagnostics. These components are important because they make the repository reproducible and allow trained models to be reused consistently across training, inference, and result generation workflows.

The user-facing scripts under `scripts/` are thin but important workflow connectors. Scripts such as `train.py`, `eval_metrics.py`, `predict_image.py`, and `benchmark_pipeline.py` do not contain the main modelling logic themselves, but they are the main entry points through which a user accesses the lower-level implementation for complete training, evaluation, inference, and benchmarking tasks.

2.6 Important Errors and Their Conditions

Input and Configuration Errors

Several important errors in the software are caused by invalid inputs, missing files, or inconsistent configuration between artefacts. For example, the checkpoint utilities raise errors when a checkpoint is missing required metadata fields, has a mismatched keypoint layout, contains duplicate keypoint indices, or uses an unsupported checkpoint version. Similar validation errors occur when invalid keypoint subsets are requested, when unsupported dataset names or split aliases are supplied, or when an invalid prediction mode or negative PCK threshold is passed on the command line. In these cases, the software responds by raising a clear `ValueError` and stopping execution rather than continuing with inconsistent state.

Runtime Resource Errors

A second group of important errors occurs when required runtime resources are not available. Examples include missing dataset directories, missing image files, empty evaluation image folders, failed imports of optional dependencies such as MediaPipe, or requesting CUDA on a machine where it is not available. These conditions are handled through exceptions such as `FileNotFoundError` or `RuntimeError`. The software does not silently ignore these problems; instead, it fails early and reports the missing dependency, missing path, or unsupported device condition so that the user can correct the environment before rerunning the workflow.

Workflow State Errors

Further errors can arise from invalid workflow state during training or evaluation. For example, stage-specific training functions raise errors if an unsupported stage number is used, if gradient accumulation is configured with an invalid value, or if stage 2 is requested without the required coordinate loss. Training also raises an error if `-skip-stage1` is used without providing an initial checkpoint.

Evaluation and qualitative-rendering workflows can also fail when no samples match the requested dataset split or when the selected checkpoint does not contain the keypoints required for shared-10 evaluation or fusion. In these situations, the software again responds by terminating with an explicit error message, which helps prevent silent misuse of the repository and makes debugging more straightforward.

2.7 User Interface Design

Command-Line Interaction

The user interacts with the software primarily through a command-line interface. The main workflows are exposed as separate Python scripts under `scripts/`, including training, evaluation, benchmarking, single-image inference, qualitative rendering, and result aggregation.

In practice, the user runs these scripts from the repository root, supplies the required paths and options as command-line arguments, and receives outputs such as checkpoints, JSON result files, CSV summaries, printed metrics, or rendered visualisations.

This command-line design was chosen because the software is a research artefact rather than an end-user application. The main tasks involve running controlled experiments, reproducing reported results, evaluating checkpoints under different conditions, and generating analysis artefacts.

These activities are better supported by explicit script arguments than by a graphical user interface, because command-line workflows are easier to document, automate, reproduce, and integrate with local machines, Conda environments, and HPC or Slurm-based execution.

Separation of Interface and Core Logic

Another important interface choice was to separate the reusable implementation from the user-facing entry points. The core logic is implemented in the `src/handpose/` package, while the scripts act as thin workflow wrappers around that logic. This was done so that the user interface remains simple at the script level while the lower-level code remains modular and reusable. As a result, users can run complete workflows through a small number of documented commands without needing to interact directly with the internal package structure.

Explicit Runtime Configuration

The interface relies on explicit file and path arguments for datasets, checkpoints, and output locations. This choice was made to keep the repository portable across different machines and storage layouts, especially since some datasets are external and not bundled with the artefact. By making these dependencies explicit at runtime, the software avoids hard-coding machine-specific paths and makes the execution process clearer for assessors and future users.

2.8 Quality Assurance Steps

Documentation and Artefact Management

Maintainability was further supported through reproducibility-oriented documentation and artefact management. The repository includes installation instructions, requirement notes, a replication guide, curated experiment command files, preserved checkpoints, and saved result artefacts. These were important quality-assurance measures because they made the software easier to inspect, rerun, and extend, while also ensuring that the reported experimental outputs could be traced back to concrete workflows within the repository.

2.8.1 Version Control and Versioning

Version control was used throughout development by maintaining the project as a Git-based repository. This allowed the software, documentation, experiment definitions, and preserved artefacts to be developed within a single structured history rather than as disconnected files. Using version control was important because the project involved repeated changes to model design, training configuration, evaluation workflows, reporting artefacts, and supporting documentation, all of which needed to remain traceable over time.

Changes were tracked and managed through the normal Git workflow of recording edits to the repository as development progressed. This made it possible to monitor the evolution of scripts, source modules, experiment files, and paper materials, and helped prevent important changes from being lost or mixed together without record. It also supported safer experimentation, since alternative implementations or revised configurations could be introduced while preserving the earlier state of the project.

2.8.2 Code Style and Convention

The code follows a consistent Python style based on clear module separation, descriptive function names, and lightweight internal documentation. The repository is organised by responsibility, with separate modules for data loading, model definition, training, inference, evaluation, and checkpoint handling. Within these modules, functions and classes generally use descriptive snake_case naming for functions and variables, and class-based naming for model components such as `HandPoseNet`. This made the code easier to navigate because related logic was grouped together rather than scattered across large scripts.

A consistent convention was also followed for docstrings and purpose statements. Most modules begin with a short summary docstring explaining their role in the repository, and most important functions include concise docstrings describing what they do. This helped keep the code clear because users and future readers can understand the role of a file or function without having to infer it entirely from implementation details.

The code also follows a convention of keeping user-facing workflow logic separate from reusable implementation logic. Scripts under `scripts/` mainly parse arguments and call functions from the `src/handpose/` package, while the package itself contains the lower-level logic. This convention improved consistency because training, evaluation, inference, and benchmarking were all built on shared internal code rather than separate duplicated implementations.

The use of explicit validation and informative error messages at key interfaces. Functions that depend on specific assumptions, such as checkpoint schema, prediction modes, dataset names, keypoint layouts, or device availability, usually check these conditions directly and raise clear exceptions if they are violated. This kept the behaviour of the software more predictable and made the codebase easier to maintain, since failures tended to occur close to the source of the problem rather than emerging later as ambiguous downstream bugs.

2.8.3 Automated Testing

The repository includes a lightweight automated test suite implemented with `pytest`. These tests provide a fast validation layer before running more expensive workflows such as full training or dataset-based evaluation.

`test_hand_pose_net_output_shapes` in `tests/test_model_shapes.py`. This test instantiates `HandPoseNet` for both the standard 21-keypoint layout and the reduced 10-keypoint layout, passes a dummy tensor of shape `(2,3,256,256)` through the model, and checks that the outputs have shapes `(2,K,64,64)` for heatmaps and `(2,K,2)` for coordinates. It also checks that the returned tensors contain only finite values. This verifies the most basic output-shape contract of the model.

`test_heatmaps_to_coords_argmax_decodes_normalized_xy` in `tests/test_fusion.py`. This test constructs a small heatmap with a known maximum location and checks that `heatmaps_to_coords_argmax` returns the expected normalized coordinate. The same file also includes `test_resolve_fusion_bone_edges_supports_tip_base_10_and_rejects_other_layouts`, which checks that supported keypoint layouts are accepted while unsupported layouts raise a `ValueError`. These tests verify both correct fusion behaviour and correct rejection of invalid inputs.

`test_validate_checkpoint_accepts_and_normalizes_valid_schema` in `tests/test_checkpoints.py`. This test supplies checkpoint metadata with string values for stage and epoch and checks that validation normalizes them correctly. The same file also includes `test_validate_checkpoint_rejects_duplicate_keypoint_indices` and `test_validate_checkpoint_rejects_inconsistent_num_keypoints`, which confirm that malformed checkpoint metadata is rejected. These tests protect the checkpoint contract used by training, evaluation, and inference workflows.